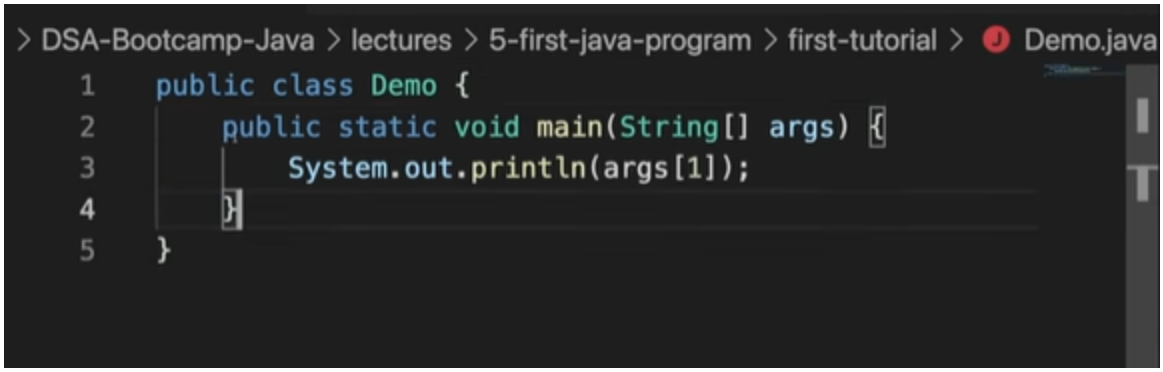
Boiler Plate →

- In Java, every file which ends with .java is a class by itself
- **Class** → is a named group of properties and functions
- In every file, the name of the class will always be the same as the name of the file. If not, the compiler throws an error.
- The whole code will be written in that class
- Class name starts with a capital letter. (Generally used convention)
- Many classes can be written in a .java file. But, the class name which is the same as the file name should always be a public class.
- **Public** → Can be seen by all. Even from other files. Can be accessed from anywhere. Called as Access modifiers.
- Inside the class, we should create a function. And the function name has to be main.
- **Function / Method** → is a block or collection of code that is reusable.
- **main** → is the function where the program starts. It is the entry point to the program. Main function is searched first, and this is the first function which to be executed
- **static** → The static variables and functions do not depend upon the objects (The instances which are created from the classes)
- Here, Demo or Main is the class name in the file. Which is not yet executed to create an instance (Object) of that class. So, as discussed above, static objects do not depend on the objects. So the first function which should run in the class should be independent of the class or object. So, that is why static is used before the main function. Or the main function is made static.

- **Void** → This is the return type of the function. Void means this doesn't return anything.
- **String[ ] args** → These are command line arguments. Which is a collection of strings.
- The arguments which we are given in the terminal are stored in the arguments as an array of strings.



- The above will print out the command line arguments provided by us in the terminal.
- Same as we print the entries of an array or characters in a string.

Javac →

```
None
javac -d . Main.java
```

- The above will save the byte code in the same folder.

```
None
javac -d .. Main.java
```

- The above will save the byte code in the previous folder.
- Generally the byte code will go to the previous folder of the source code (src) folder. Which is good practice.
- Where javac, java, git, python are located in the computer.

```
None
where javac
where java
where git
where python
```

- The above commands will give the address or path of the directory as where they are located in the computer.

```
None
ls /usr/bin
```

- Gives us the list of the files contained in this folder.
- All these files will be the .exe files.

None

```
ls /usr/bin | grep javac
```

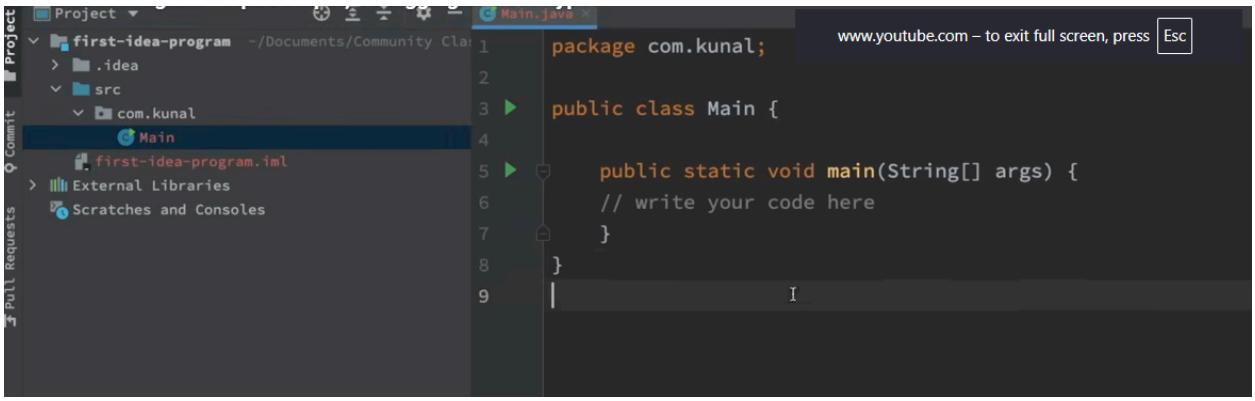
- The above applies filter to the command like if javac exists……
- When we execute the command in the command line, the system will search if the git or javac or java file exists or not in the system though the path variables and environment variables.

None

```
echo $PATH
```

- The above command will show all the path variables. i. e., it shows all the paths where the system searched for to find the file. When it finds the file, it will stop searching and start executing.

IntellijIDEA →



- The above is an IntellijIDEA screen. It will show the **selected folder**, in the folder, we can see the src file.
- **Src** → file contains the source code and the package.
- We can make a class public, only when the name of the file matches with the name of the class. Other classes cannot be made public in a file.

Java

```
// The file MUST be named MainApp.java because this class is public
public class MainApp {
    public static void main(String[] args) {
        HelperTool tool = new HelperTool();
        tool.displayMessage();
    }
}
```

```
// This additional class cannot be public
class HelperTool {
    void displayMessage() {
        System.out.println("Hello from the helper class!");
    }
}
//for further reference
```

- **Package** → is a folder in which a java file lies. So, the files in this package or folder are unable to be accessed by the files in other folders.
- This is like an extra layer of security. Access modifiers. Fully discussed during OOPs concept.
- Java contains some basic methods or functions which we can use foe example: input, output,debugging, print, etc.
- The lang package contains all these functions. This is inherited by default.
- Ctrl + click (or) Cmd + click to navigate to that class where the function is stored.

Output →

- So, the **system class** has a variable called **out** (which is of printstream). And out has a method called println()
- Out is of printstream and println will be in printstream.
- More about this in OOP.

```
Java
System.out.println("Hello from the helper class!")
```

- The above is used for printing out the result on the command line.
- println() → adds new line at the end.
- print(0 –. Doesnt add a new line at the end.

Input →

- Input is taken into the system using scanner class.
- Scanner is available in the java.util package.

```
Java
Scanner input = new scanner() //provide the object inside () from where input is being taken from
Scanner input = new scanner(System.in) // As shown here System.in means input is from the keyboard. by making changes, we can make it a text file and so on...
```

- System.out → is the standard output stream. Command line is the standard out stream In that standard output stream, println(“....something….”).
- If out is assigned to a file, then the println(“....something….”) will add ….something…. To that file.
- But, for input we have to create a new variable with new keyword and initialize it with a constructor.
- System.in → is the standard output stream. Which is the keyboard. Initially it is also null.
- If system.in is replaced with a file name, then it reads input from that file.
- Basically if before a variable, scanner is written, that means the variable will read what we want to read.
- The new Scanner(system.in) object will read input from system.in or keyboard. If it is assigned to a file then it will read from that file.
- 44:00